

Chapitre 4

Simulations

Comment sont effectuées les simulations de lois de probabilités ? Dans les logiciels de calculs, de nombreuses lois ont une fonction prédéfinie appropriée ... Mais, si elle n'est pas définie, il faut savoir programmer la bonne fonction.

4.1 Aléatoires ?

On a une notion intuitive de ce qu'est un résultat aléatoire (imprévisible?), mais pas de formulation mathématique claire. Généralement, on demande aux suites de nombres aléatoires de répondre à un certain nombre de contraintes (nombreuses, en théorie infinies). Nous nous limiterons aux suites de nombres *pseudo-aléatoires* qui sont les suites fournies par les logiciels informatiques.

Exemple 4.1 – Décimales de π

Ainsi, les décimales de π semblent se répartir à peu près uniformément sur les valeurs $\llbracket 0, 9 \rrbracket$, comme le montre la session `Wxmaxima ??`, page ??.

4.1.1 Nombres pseudo-aléatoires

La génération des nombres aléatoires (et l'absence d'une bonne définition) nous oblige à trouver des suites qui remplissent approximativement le rôle voulu. Ce sont les nombres *pseudo-aléatoires*. De manière assez étrange, les générateurs les plus utilisés viennent de l'algèbre générale...

La qualité des générateurs de nombres pseudo-aléatoires est mesurée à l'aide de différents tests. On peut consulter le livre [3] de Donald E. Knuth, pour une approche mathématique ou la site <http://simul.iro.umontreal.ca/testu01/tu01.html> pour une approche plus expérimentale.

Exemple 4.2 – Méthode de Lehmer

Soit $x_0 \in \mathbb{N}$ (valeur appelée la *graine* – *seed* en anglais), et trois nombres $(a, M) \in \mathbb{N}^{*2}$ et $b \in \mathbb{N}$, on considère la suite récurrente définie par (congruence *modulo* M)

$$\forall n \in \mathbb{N}, x_{n+1} = ax_n + b \pmod{M}$$

On trouve, par exemple (voir https://fr.wikipedia.org/wiki/G%C3%A9n%C3%A9rateur_congruentiel_lin%C3%A9aire).

1. Langage $\mathcal{C}$

$$M = 2^{31}, \quad a = 1103515245 \quad \text{et} \quad b = 12345$$

Il est conseillé de ne prendre que les 16 premiers bits (de poids fort).

2. Haynes

$$M = 2^{64}, \quad a = 6364136223846793005 \quad \text{et} \quad b = 0$$

Exemple 4.3 – Mersenne twister

Ce générateur utilise un nombre de Mersenne premier $2^{19937} - 1$. L'algorithme peut être trouvé à la page https://en.wikipedia.org/wiki/Mersenne_Twister. Il est utilisé dans de nombreux langages (notamment Python).

Exercice(s) 4.1

4.1.1 Utiliser les deux générateurs linéaires de l'exemple 4.2, de la présente page pour faire un tirage uniforme sur

- (a) le segment $[0, 1]$;
- (b) le carré $[0, 1]^2$

et comparer avec ce qu'on obtient avec le générateur aléatoire de Python. Conclusion.

4.1.2 Principe de simulation

Proposition 4.1

Soit X une v.a.r. définie sur un espace probabilisé $(\Omega, \mathcal{T}, \mathbb{P})$, de fonction de répartition F_X . On pose

$$\forall x \in]0, 1[, G(x) = \inf \left(\{u \in \mathbb{R}, F_X(u) \geq x\} \right)$$

Soit, de plus, U une v.a.r. suivant une loi $\mathcal{U}(0, 1)$, alors

$G(U)$ a même loi que X

Démonstration

Soit $x \in \mathbb{R}$ et $u \in]0, 1[$, alors

$$[G(u) \leq x] \iff [u \leq F_X(x)]$$

En effet, posons pour $u \in]0, 1[$,

$$\Delta_u = \{y \in \mathbb{R}, F_X(y) \geq u\}$$

Comme F_X est croissante, Δ_u est un intervalle de la forme $]G(u), +\infty[$ ou $[G(u), +\infty[$.

1. ($\Leftarrow$) Si $u \leq F_X(x)$, cela signifie que $x \in \Delta_u$, or $G(u) = \inf(\Delta_u)$ donc, bien évidemment $x \geq G(u)$.
2. ($\Rightarrow$) Si $G(u) \leq x$, alors pour tout $n \in \mathbb{N}^*$, $x + 1/n \in \Delta_u$, ce qui signifie que

$$F_X\left(x + \frac{1}{n}\right) \geq u$$

ce qui, par continuité à droite de F_X nous donne, en faisant tendre n vers $+\infty$, $F_X(x) \geq u$.

Finalement, pour $x \in \mathbb{R}$

$$\mathbb{P}(G(U) \leq x) = \mathbb{P}(U \leq F_X(x)) = F_X(x)$$

■

Remarque importante 4.1

Cette méthode est très pratique pour obtenir une v.a.r. de même loi que X à condition qu'on soit capable de calculer G . Par exemple, lorsque F_X

admet une réciproque (continue, strictement croissante), alors $G = F_X^{-1}$.

4.2 V.a.r. discrètes

4.2.1 Loix finies

Propriété 4.1 – Simulation d’une loi finie

On suppose ici que $X : (\Omega, \mathcal{T}, \mathbb{P}) \longrightarrow \mathbb{R}$ est une v.a.r., telle que

$$X(\Omega) = \llbracket 1, n \rrbracket \quad \text{et} \quad \forall k \in \llbracket 1, n \rrbracket, \mathbb{P}(X = k) = p_k \in]0, 1[$$

- On découpe alors $[0, 1]$ en une réunion d’intervalles de longueurs p_k , de la forme $[u_k, u_{k+1}]$, où $u_0 = 0$ et

$$\forall k \in \llbracket 1, n \rrbracket, u_k - u_{k-1} = p_k$$

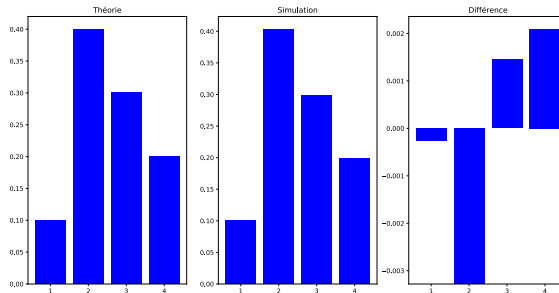
On peut alors simuler X à partir d’une loi uniforme $U \sim \mathcal{U}(0, 1)$, en prenant ^a

$$[(X = k)] \iff [U \in [u_{k-1}, u_k[$$

Voir le code Python ??, page ?? et la figure 4.1, de la présente page

a. C’est la méthode vue à la proposition 4.1, page précédente.

Figure 4.1 – Simulation d’une loi discrète (cas fini)



4.2.2 Lois infinies

Propriété 4.2 – Simulation d’une loi discrète infinie

On suppose ici que

$$X(\Omega) = \mathbb{N} \quad \text{et} \quad \forall k \in \mathbb{N}, \mathbb{P}(X = k) = p_k \in [0, 1]$$

- On peut procéder comme ci-dessus, mais cela demande de calculer les u_k , pour k éventuellement très grand, lorsque $U(\omega)$ sera très proche de 1. En revanche, quand on connaît les u_k , c’est une méthode appropriée. Par exemple, pour une loi géométrique de paramètre p , on a

$$u_0 = u_1 = 0 \quad \text{et} \quad \forall k \in \mathbb{N}^*, u_{k+1} = \sum_{j=1}^k p(1-p)^{j-1} = 1 - (1-p)^k$$

On a donc la simulation

$$\forall k \in \mathbb{N}^*, [(X = k)] \iff \left[k = 1 + \left\lceil \frac{\ln(1-U)}{\ln(1-p)} \right\rceil \right]$$

Le code **Python ??**, page ?? et la figure 4.2, page suivante sont une illustration de cette technique.^a

a. Pour la loi géométrique, on peut aussi utiliser le fait que, si $X \sim \mathcal{E}(\lambda)$, alors

$$[X] \sim \mathcal{G}(1 - e^{-\lambda})$$

Mais, c’est un cas particulier, où la loi discrète qui nous occupe s’obtient à partir d’une loi facile à simuler.

Propriété 4.3 – Simulation d’une loi discrète infinie (suite)

- On peut aussi utiliser des résultats sur ces lois. Par exemple, on a vu d’où venait l’utilisation des lois de Poisson pour modéliser une file d’attente (exemple 6.2, page 155 de [1]). Comme il est facile de simuler une loi exponentielle, on procédera de la manière suivante.

1. On réalise des **v.a.r.** $(T_n)_{n \in \mathbb{N}}$ indépendantes de même loi exponentielle $\mathcal{E}(1)$, et on définit une simulation de $X \sim \mathcal{P}(\lambda)$ par

$$[(X = k)] \iff \left[\sum_{j=1}^k T_j \leq \lambda \quad \text{et} \quad \sum_{j=1}^{k+1} T_j > \lambda \right]$$

Le code **Python ??**, page ?? et la figure 4.3, page suivante sont

une illustration de cette technique.

Figure 4.2 – Simulation d’une loi géométrique

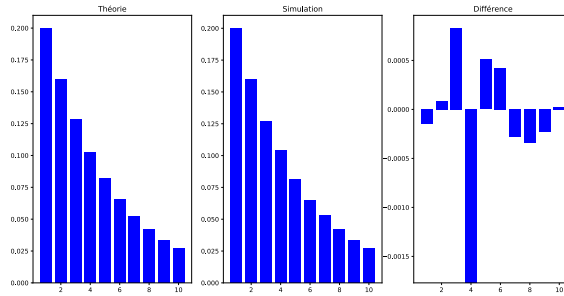
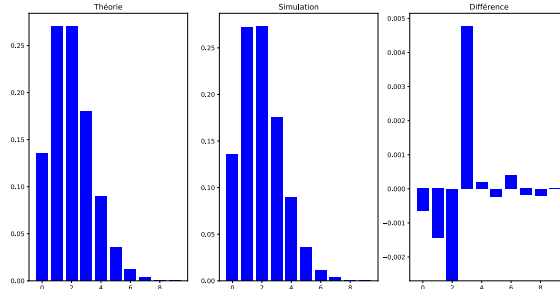


Figure 4.3 – Simulation d’une loi de Poisson



4.2.3 Lois conjointes

Propriété 4.4 – Simulation de variables aléatoires à valeurs dans $\mathbb{R}^p$

Soit $X = (X_1, \dots, X_p)$ une variable aléatoire définie sur $(\Omega, \mathcal{T}, \mathbb{P})$ à valeurs dans $\mathbb{R}^p$.

1. Si les lois marginales sont indépendantes, on simule chaque loi marginale...
2. Cas fini

On suppose donc que $X(\Omega) = \prod_{j=1}^p \llbracket 1, n_j \rrbracket$. Et on connaît

$$\forall (k_1, \dots, k_p) \in \prod_{j=1}^p \llbracket 1, n_j \rrbracket, \mathbb{P}((X_1 = k_1) \cap \dots \cap (X_p = k_p))$$

On utilise le même principe que pour les v.a.r. en ordonnant $X(\Omega)$, avec un ordre lexicographique

$$x_1 = (1, \dots, 1, 1) < x_2 = (1, \dots, 1, 2) < \dots < x_{n_p} = (1, \dots, 1, n_p) < \\ x_{n_p+1} = (1, \dots, 2, 1) < \dots < x_{2n_p} = (1, \dots, 2, n_p) < \dots < x_{n_1 \dots n_p} = (n_1, \dots, n_p)$$

Ainsi, pour $p = 3$, $n_1 = 2$, $n_2 = 3$ et $n_3 = 4$, on obtient
 $(1, 1, 1) < (1, 1, 2) < (1, 1, 3) < (1, 1, 4) < (1, 2, 1) < (1, 2, 2) < (1, 2, 3) < (1, 2, 4) < (1, 3, 1) < (1, 3, 2) < (1, 3, 3) < (1, 3, 4) < (2, 1, 1) < (2, 1, 2) < (2, 1, 3) < (2, 1, 4) < (2, 2, 1) < (2, 2, 2) < (2, 2, 3) < (2, 2, 4) < (2, 3, 1) < (2, 3, 2) < (2, 3, 3) < (2, 3, 4)$. Et on simule par

$$[(Y = k)] \iff [(X = x_k)]$$

On peut utiliser aussi une méthode inspirée de la deuxième méthode de rejet, voir page 140.

3. Cas infini

On applique le même principe, en mettant un ordre arbitraire sur $X(\Omega)$.^a

a. On peut se rappeler comment nous avons numéroté $\mathbb{N}^2$ en première année.

4.3 V.a.r. continues

Propriété 4.5 – Simulation d'une v.a.r. continue

Soit $X : (\Omega, \mathcal{T}, \mathbb{P}) \longrightarrow \mathbb{R}$, une variable aléatoire continue, on utilise alors, si l'on connaît la fonction de répartition F_X la méthode de la proposition 4.1, page 129.

- *Loi uniforme sur $[a, b]$.*

Il suffit alors d'utiliser

$$[U \sim \mathcal{U}(0, 1)] \implies [a + (b - a)U \sim \mathcal{U}(a, b)]$$

Le code **Python** ??, page ?? et la figure 4.4, page suivante sont une illustration de cette technique.

- *Loi exponentielle de paramètre $\lambda > 0$.*

La fonction de répartition est alors

$$x \mapsto (1 - e^{-\lambda x}) \mathbb{1}_{\mathbb{R}_+}(x)$$

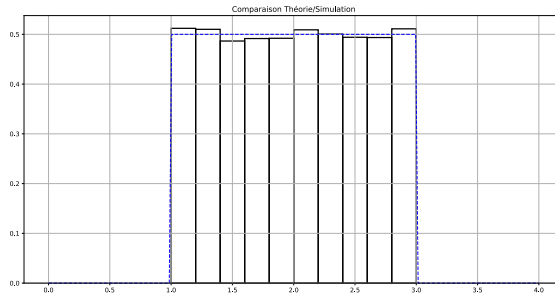
On a donc ^a

$$[U \sim \mathcal{U}(0, 1)] \implies \left[-\frac{\ln(1 - U)}{\lambda} \sim \mathcal{E}(\lambda) \right]$$

Le code `Python ??`, page ?? et le figure 4.5, page suivante sont une illustration de cette méthode.

a. Comme $1 - U$ suit aussi une loi uniforme sur $[0, 1]$, on peut utiliser $-\ln(U)/\lambda$.

Figure 4.4 – Simulation d’une loi uniforme



Propriété 4.6 – Simulation d’une loi normale

- Loi normale de paramètres m et σ^2 .

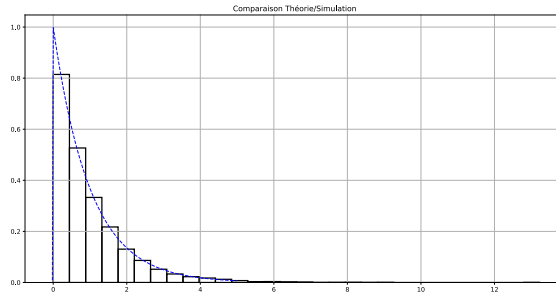
On a d’abord

$$\left[X \sim \mathcal{N}(m, \sigma^2) \right] \iff \left[\frac{X - m}{\sigma} \sim \mathcal{N}(0, 1) \right]$$

Nous allons donc d’abord simuler une v.a.r. suivant une loi normale centrée réduite.

Nous ne connaissons pas bien la fonction de répartition de la loi normale centrée réduite, en particulier pas sa fonction réciproque. Nous allons donc utiliser l’exemple 6.3, page 156 de [1]. En faisant le calcul

Figure 4.5 – Simulation d’une loi exponentielle



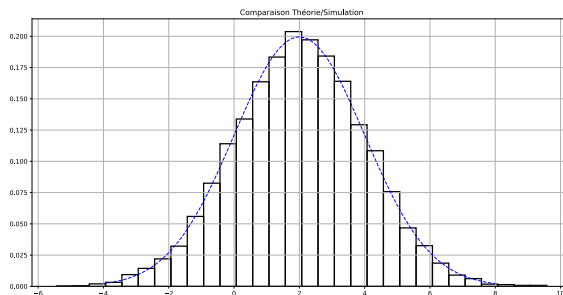
inverse, nous voyons que

$$\left[R^2 \sim \mathcal{E}\left(\frac{1}{2}\right) \quad \text{et} \quad \Theta \sim \mathcal{U}(0, 2\pi) \right] \iff \left[R \cos(\Theta) \sim \mathcal{N}(0, 1) \quad \text{et} \quad R \sin(\Theta) \sim \mathcal{N}(0, 1) \right]$$

De plus, les deux lois normales centrées réduites obtenues sont indépendantes... On en génère deux d’un coup !

Le code `Python ??`, page ?? et la figure 4.6, de la présente page une illustration de cette méthode.

Figure 4.6 – Simulation d’une loi normale



4.4 Loi uniforme

Rappel 4.1 – Loi uniforme

Soit $(\Omega, \mathcal{T}, \mathbb{P})$ un espace probabilisé, soit $B \in \mathcal{BO}(\mathbb{R}^p)$ un borélien de $\mathbb{R}^p$ de mesure de Lebesgue finie et non nulle ($\lambda(B) \in]0, +\infty[$). On dit qu'une variable aléatoire suit une loi uniforme sur B si

$$\forall A \in \mathcal{BO}(\mathbb{R}^p), \mathbb{P}(X \in A) = \frac{\lambda(A \cap B)}{\lambda(B)}$$

On note

$$X \sim \mathcal{U}(B).$$

Cette loi modélise un tirage « au hasard » dans B .

Démonstration de la bonne définition de la loi uniforme

C'est bien une loi de probabilités, car $\mathbb{P}(X \in \mathbb{R}^2) = \frac{\lambda(B)}{\lambda(B)} = 1$. ■

Exemple 4.4

1. Nous avons déjà rencontré le cas où $B = [0, 1]^2 \subset \mathbb{R}^2$, et démontré (voir l'exemple 6.4, page 169 de [1]) que X et Y (coordonnées du point dans le repère euclidien canonique) étaient indépendantes de loi uniforme sur $[0, 1]$.
2. Nous avons aussi rencontré le cas où $B = \{(x, y) \in \mathbb{R}^2, 0 \leq x, 0 \leq y \text{ et } x + y \leq 1\}$ (voir l'exemple 6.5, page 169 de [1]). Les coordonnées du point n'étaient plus des v.a.r. indépendantes.

Exemple 4.5 – Loi uniforme sur un disque

Lorsque B est le disque de centre O et de rayon 1, alors $R = \sqrt{X^2 + Y^2}$ et Θ à valeurs dans $[0, 2\pi[$ sont indépendantes et, de plus (voir le code `Python ??`, page ?? qui génère la figure 4.7, page suivante, à gauche)

$$R^2 \sim \mathcal{U}(0, 1) \quad \text{et} \quad \Theta \sim \mathcal{U}(0, 2\pi)$$

Notons que c'est le carré du rayon qui suit une loi uniforme et non le rayon qui donne la mauvaise simulation illustrée par la figure 4.7, page ci-contre, à droite générée par le code `Python ??`, page ??.

Démonstration

Pour $r_2 \in]0, 1[$ et $\theta \in]0, 2\pi[$, on a

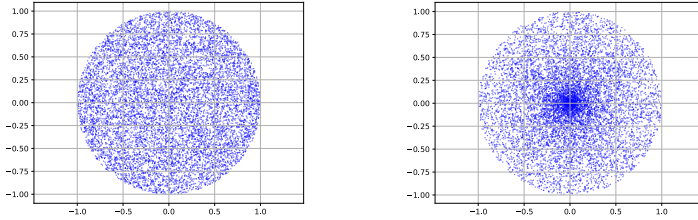
$$\mathbb{P}\left(R^2 \leq r_2, \Theta \leq \theta\right) = \frac{\theta}{2\pi} r_2$$

car l'aire du secteur défini pour $\rho_0 > 0$ et $\theta_0 \in [0, 2\pi]$ par

$$\left\{(\rho, \theta) \in \mathbb{R}^2, \rho \in [0, \rho_0] \text{ et } \theta \in [0, \theta_0]\right\}$$

a pour aire $\rho_0^2 \theta_0 / (2\pi)$. ■

Figure 4.7 – Loi uniforme sur un disque (bonne simulation à gauche, mauvaise à droite)



4.5 Méthodes de rejet

4.5.1 V.a.r. discrètes

Propriété 4.7 – Méthode du rejet discrète

On suppose $X(\Omega) = \mathbb{N}$, et

$$\forall n \in \mathbb{N}, \mathbb{P}(X = n) = p_n \in [0, 1], \quad \text{avec} \quad \sum_{n=0}^{+\infty} p_n = 1$$

Soit $(q_n)_{n \in \mathbb{N}}$ définissant une autre loi de probabilité Y (que l'on sait mieux simuler), et on suppose connue une constante $k > 0$, telle que

$$\forall n \in \mathbb{N}, p_n \leq k q_n$$

On procède alors comme suit

- On tire un couple $(U(\omega), Y(\omega))$, où $U \sim \mathcal{U}(0, 1)$ est indépendante de Y (notons $n = Y(\omega)$).

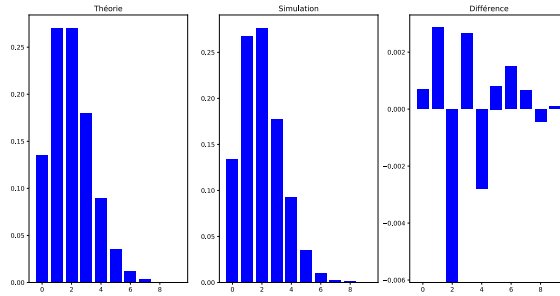
- Si $k U(\omega) q_n \leq p_n$, on accepte la valeur.
- Sinon, on tire un nouveau couple, etc.

Les valeurs trouvées sont des réalisations d'une v.a.r. de même loi que X . Le code Python ??, page ?? et la figure 4.8, de la présente page sont une illustration de cette méthode. (Simulation d'une loi de Poisson de paramètre λ).

Démonstration

Voir la démonstration du théorème 4.1, de la présente page. ■

Figure 4.8 – Simulation d'une loi de Poisson (méthode du rejet)



4.5.2 Variables continues

Théorème 4.1 – Méthode du rejet (première version)

Soit B un borélien de $\mathbb{R}^2$, borné, de mesure de Lebesgue non nulle.

$$B \subset \prod_{k=1}^p [a_k, b_k] \stackrel{\text{Not}}{=} \Pi$$

où Π est donc un pavé contenant B . Soit $U \sim \mathcal{U}(\Pi)$ une variable aléatoire suivant une loi uniforme sur Π . Et soit $(U_n)_{n \in \mathbb{N}^*}$ une suite de variables aléatoires indépendantes de même loi que U , on pose

$$N = \min \left(\{k \in \mathbb{N}^*, U_k \in B\} \right) \text{ si l'ensemble est non vide, } 1 \text{ sinon}$$

alors N suit une loi géométrique de paramètre $\lambda(B)/\lambda(\Pi)$ et

$$U_N \sim \mathcal{U}(B)$$

Démonstration

1. La probabilité que $U \in B$, est, par définition d'une loi uniforme

$$\mathbb{P}(U \in B) = \frac{\lambda(B \cap \Pi)}{\lambda(\Pi)} = p, \quad \text{où } \lambda \text{ est la mesure de Lebesgue}$$

On peut remarquer par ailleurs que

$$\mathbb{P}\left(\left\{k \in \mathbb{N}^*, U_k \in B\right\} = \emptyset\right) = 0$$

car

$$\left(\left\{k \in \mathbb{N}^*, U_k \in B\right\} = \emptyset\right) = \bigcap_{k \in \mathbb{N}^*} (U_k \notin B)$$

et ces événements sont indépendants, donc

$$\mathbb{P}\left(\bigcap_{k \in \mathbb{N}^*} (U_k \notin B)\right) = \lim_{n \rightarrow +\infty} (1-p)^n = 0$$

car $p \neq 0$, d'après l'hypothèse $\lambda(B) \neq 0$.

N suit donc bien une loi géométrique de paramètre $p = \lambda(B)/\lambda(\Pi)$.

2. Calculons la loi de U_N . D'après la loi des probabilités totales, on a, pour $A \in \mathcal{BO}(\mathbb{R}^p)$

$$\mathbb{P}(U_N \in A) = \sum_{k=1}^{+\infty} \mathbb{P}\left((U_N \in A) \cap (N = k)\right)$$

Or

$$(U_N \in A) \cap (N = k) = (U_k \in A \cap B) \cap \left(\bigcap_{j=1}^{k-1} (U_j \notin B)\right)$$

de probabilité immédiate (par indépendance des U_k)

$$\frac{\lambda(A \cap B)}{\lambda(\Pi)} (1-p)^{k-1}$$

Finalement

$$\mathbb{P}(U_N \in A) = \frac{\lambda(A \cap B)}{\lambda(\Pi)} \sum_{k=1}^{+\infty} (1-p)^{k-1} = \frac{\lambda(A \cap B)}{\lambda(\Pi)} \frac{1}{p} = \frac{\lambda(A \cap B)}{\lambda(B)}$$

■

Remarque 4.2

Cela s'appelle méthode du rejet, car, lorsque le point tiré *n'est pas dans B*, on recommence le tirage.

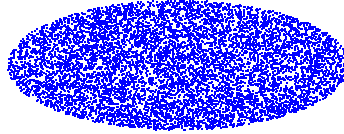
Remarque importante 4.3

Pour tirer le moins possible de points inutiles, il faut que la probabilité de succès p soit la plus proche de 1, et donc choisir Π le plus petit possible...

Exercice(s) 4.2

4.2.1 En utilisant la méthode du rejet, simuler une loi uniforme sur l'intérieur de l'ellipse d'équation $x^2 + 3y^2 = 1$. Voir la figure 4.9, de la présente page.

Figure 4.9 – Loi uniforme sur une ellipse



Théorème 4.2 – Méthode du rejet (deuxième version)

Soit X une variable aléatoire définie sur un espace probabilisé $(\Omega, \mathcal{T}, \mathbb{P})$ à valeurs dans $\mathbb{R}^p$, dont on connaît la densité $f_X : \mathbb{R}^p \rightarrow \mathbb{R}_+$. On suppose connue une densité $f_Y : \mathbb{R}^p \rightarrow \mathbb{R}_+$, d'une variable aléatoire Y , telle que

$$\exists k \in \mathbb{R}_+^*, \forall \underline{x} \in \mathbb{R}^p, f_X(\underline{x}) \leq k f_Y(\underline{x})$$

Soit $(Y_n)_{n \in \mathbb{N}^*}$ des variables aléatoires indépendantes de loi de densité f_Y , et $(U_n)_{n \in \mathbb{N}^*}$ des *v.a.r.* indépendantes, de lois uniformes sur $[0, 1]$ et indépendantes des $(Y_n)_{n \in \mathbb{N}^*}$, alors

$N = \min \left(\{j \in \mathbb{N}^*, k U_j f_Y(Y_j) \leq f_X(Y_j)\} \right)$, si l'ensemble est non vide, 1 sinon suit une loi géométrique de paramètre $1/k$ et

Y_N a même loi que X

Démonstration

1. Calculons la probabilité δ que

$$(U, Y) \in \left\{ (u, \underline{y}) \in [0, 1] \times \mathbb{R}^p, k u f_Y(\underline{y}) \leq f_X(\underline{y}) \right\}$$

C'est, par définition de la densité

$$\delta = \int_{y_1=-\infty}^{+\infty} \cdots \int_{y_p=-\infty}^{+\infty} \int_{u=0}^{\min\left(1, f_X(\underline{y}) / (k f_Y(\underline{y}))\right)} f_Y(\underline{y}) \, du \, dy_p \cdots dy_1$$

L'hypothèse faite sur les densités, nous indique, que lorsque $f_Y(\underline{y}) \neq 0$, on a ^a

$$\min\left(1, \frac{f_X(\underline{y})}{k f_Y(\underline{y})}\right) = \frac{f_X(\underline{y})}{k f_Y(\underline{y})}$$

donc

$$\delta = \frac{1}{k} \int_{\mathbb{R}^p} f_Y \, d\lambda = \frac{1}{k}$$

Le même raisonnement qu'à la démonstration précédente nous assure que $N \sim \mathcal{G}(\delta)$.

2. Calculons maintenant la loi de Y_N . Soit $(t_1, \dots, t_p) \in \mathbb{R}^p$, on a alors

$$\mathbb{P}\left(Y_N \in \prod_{j=1}^p]-\infty, t_j]\right) = \sum_{n=1}^{+\infty} \mathbb{P}\left(\left(Y_N \in \prod_{j=1}^p]-\infty, t_j]\right) \cap (N = n)\right) = \sum_{n=1}^{+\infty} \mathbb{P}\left(\left(Y_N \in \prod_{j=1}^p]-\infty, t_j]\right) \cap (N = n)\right)$$

Or, pour $n \in \mathbb{N}^*$, on a

$$q_n = \mathbb{P}\left(\left(Y_N \in \prod_{j=1}^p]-\infty, t_j]\right) \cap (N = n)\right) = \mathbb{P}\left(\left(\bigcap_{j=1}^{n-1} (U_j, Y_j) \in \Delta\right) \cap ((U_n, Y_n) \in \Gamma)\right)$$

où

$$\Delta = \left\{ (u, \underline{y}) \in [0, 1] \times \mathbb{R}^p, k u f_Y(\underline{y}) > f_X(\underline{y}) \right\}$$

et

$$\Gamma = \left\{ (u, \underline{y}) \in [0, 1] \times \prod_{j=1}^p]-\infty, t_j], k u f_Y(\underline{y}) \leq f_X(\underline{y}) \right\}$$

Et, de plus, les $(U_k, Y_k)_{k \in \mathbb{N}^*}$ sont indépendantes... Donc

$$q_n = \left(1 - \frac{1}{k}\right)^{n-1} \int_{y_1=-\infty}^{+\infty} \cdots \int_{y_p=-\infty}^{+\infty} \int_{u=0}^{\min\left(1, f_X(\underline{y}) / (k f_Y(\underline{y}))\right)} f_Y(\underline{y}) \, du \, dy_p \cdots dy_1$$

Finalement

$$q_n = \left(1 - \frac{1}{k}\right)^{n-1} \frac{1}{k} \mathbb{P}\left(X \in \prod_{j=1}^p]-\infty, t_j]\right)$$

Il est alors immédiat que

$$\mathbb{P}\left(Y_N \in \prod_{j=1}^p]-\infty, t_j]\right) = \mathbb{P}\left(X \in \prod_{j=1}^p]-\infty, t_j]\right)$$

■

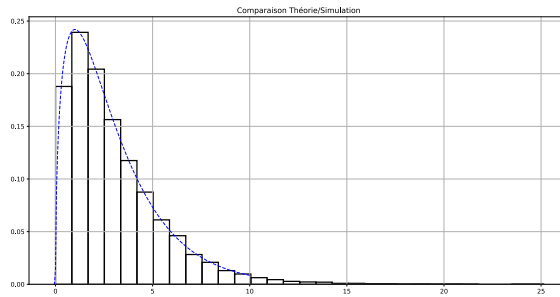
a. Et lorsque cette valeur est nulle, cela ne nous intéresse pas !

Propriété 4.8 – Simulation d’une loi du χ^2

Pour la loi du χ_n^2 , il suffit de s’appuyer, soit sur une somme de n carrés de v.a.r. indépendantes de lois normales centrées réduites. Mais, pour n « grand », cela risque d’être peu efficace, on peut faire alors appel à la *deuxième méthode du rejet*. Le code Python ??, page ?? et la figure 4.10, de la présente page sont une illustration de la première méthode.

Le code Python ??, page ?? et la figure 4.12, page suivante sont une illustration de la deuxième méthode.

Figure 4.10 – Simulation d’un χ^2



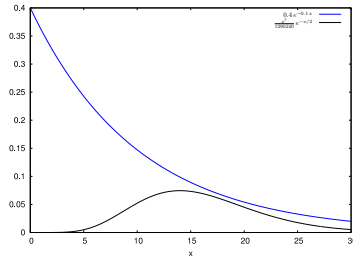
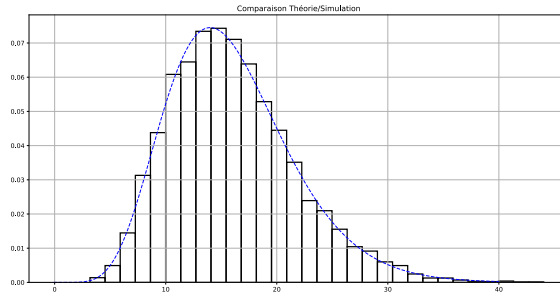
Exemple 4.6 – Simulation d’une loi du χ_{16}^2

Utilisons la deuxième méthode du rejet pour simuler une loi du χ_{16}^2 (χ^2 à 16 degrés de liberté). La densité de cette loi est

$$x \mapsto \frac{1}{1290240} x^7 e^{-x/2} \mathbb{1}_{\mathbb{R}_+^*}(x)$$

- La première étape consiste à chercher une « bonne » loi de Y (qui doit être facilement simulable, sinon, la méthode n’a pas d’intérêt). Vu l’allure de la densité du χ^2 , on pense à une loi exponentielle. Après essai, on peut prendre $\lambda = 0.1$ et $k = 4$. Voir la figure 4.11, page ci-contre.
- Ensuite, il suffit de se laisser porter par l’algorithme. Le code Python ??, page ?? et la figure 4.12, page ci-contre sont une illustration de la méthode.

Figure 4.11 – Choix d’une autre densité

Figure 4.12 – Méthode du rejet (χ^2_{16})

Exercice(s) 4.3

4.3.1 Proposer une simulation pour les lois suivantes

(a) Loi de Cauchy, de densité

$$x \mapsto \frac{1}{\pi} \frac{1}{1+x^2}$$

(b) Loi uniforme sur un pentagone régulier.

(c) Loi définie par

$$\forall n \in \mathbb{N}^*, \mathbb{P}(X = n) = \frac{6}{\pi^2} \frac{1}{n^2}$$

(d) Loi uniforme sur

$$\left\{ (x, y) \in \mathbb{R}^2, x \geq 1, y \geq 0 \text{ et } x^2 y \leq 1 \right\}$$

- 4.3.2 Utiliser la première méthode du rejet pour simuler une loi uniforme sur la boule de centre O et de rayon 1 dans $\mathbb{R}^3$.
- 4.3.3 On veut simuler une loi du χ^2 à 3 degrés de liberté par la méthode du rejet, en s'appuyant sur une loi exponentielle de paramètre λ . Quel est le meilleur choix de λ (celui qui provoquera, en moyenne, le moins de rejets).
- 4.3.4 Mettre en place une méthode du rejet pour simuler une loi normale centrée réduite.
- 4.3.5 Soit X une v.a.r. de fonction de répartition F_x . On supposera cette fonction de répartition inversible. Soit $\beta \in \mathbb{R}$, on veut simuler la loi de

$$Y = X/(X > \beta)$$

- (a) Comment simuler Y par la méthode du rejet ?

On pose, si $U \sim \mathcal{U}(0, 1)$

$$Z = F_x^{-1}(F_x(\beta) + (1 - F_x(\beta))U)$$

Quelle est la loi de Z ? En déduire une autre méthode de simulation de Y .

- (b) Si $X \sim \mathcal{N}(m, \sigma^2)$, montrer que l'on peut ramener la situation à $m = 0$, $\sigma = 1$ et $\beta \in \mathbb{R}$ quelconque.
- (c) Proposer, dans le cas précédent une simulation par la méthode du rejet en s'appuyant sur la loi de densité

$$x \longmapsto \lambda e^{-\lambda(x-\beta)} \mathbf{1}_{(x>\beta)}$$

En choisissant le paramètre λ de manière optimale.

- 4.3.6 Soit $M = (X, Y, Z)$ suivant une loi uniforme sur la boule unité B de $\mathbb{R}^3$, où

$$B = \{(x, y, z) \in \mathbb{R}^3, x^2 + y^2 + z^2 \leq 1\}$$

Quelles sont les lois de

$$R = \sqrt{X^2 + Y^2 + Z^2}, \quad \Theta \in [0, \pi] \text{ et } \Phi \in [0, 2\pi[$$

telles que

$$X = R \sin(\Theta) \cos(\Phi), \quad Y = R \sin(\Theta) \sin(\Phi) \text{ et } Z = R \cos(\Theta)$$

- 4.3.7 On veut générer de manière uniforme des points aléatoires $M = (X, Y, Z)$ sur une sphère de centre O et de rayon 1, donc pour toute partie borélienne A de la sphère B , la probabilité que $M \in A$ sera

$$\mathbb{P}(M \in A) = \frac{\mathcal{A}(A)}{\mathcal{A}(B)} = \frac{\mathcal{A}(A)}{4\pi}$$

où $\mathcal{A}(A)$ désigne l'aire de A . Quelles sont les lois de

$$\Theta \in [0, \pi] \text{ et } \Phi \in [0, 2\pi[$$

telles que

$$X = \sin(\Theta) \cos(\Phi), Y = \sin(\Theta) \sin(\Phi) \text{ et } Z = \cos(\Theta)$$

Proposer une simulation uniforme sur la sphère de centre O de rayon 1.

4.6 Utilisation de la bibliothèque `scipy.stats`

Bien sûr, il est souvent plus facile d'utiliser des outils de simulation tout programmés. C'est l'objet de la bibliothèque `scipy.stats`. Les méthodes que nous avons vues nous seront utiles pour simuler des lois de probabilités qui *ne sont pas* dans cette bibliothèque.

Session Python 4.1 – Les fonctions fondamentales de la bibliothèque

Il y a beaucoup de fonctions dans cette bibliothèque, en particulier un grand nombre de lois plus ou moins connues. Voici l'essentiel des fonctions qui nous intéresseront.

4.6.1 Lois discrètes

4.6.1.1 Création de v.a.r.

In[1]

```
1 from scipy.stats import binom, poisson, geom, randint
```

In[2]

```
1 # Création d'une  $\mathcal{B}(10, 0.3)$ 
2 X = binom(10, 0.3)
```

In[3]

```
1 # Création d'une  $\mathcal{P}(2)$ !
2 Y = poisson(2)
```

In[4]

```
1 # Création d'une  $\mathcal{G}(0.4)$ !
2 Z = geom(0.4)
```

In[5]

```
1 # Création d'une loi uniforme sur  $\left[4, 7\right]$ 
  ↳ \textdbend!
2 T = randint(4, 8)
```

4.6.1.2 Utilisation particulière des lois discrètes

In[6]

```
1 # Calcul des probabilités
2 X.pmf(2), Y.pmf(1), Z.pmf(0), T.pmf(7)
```

Out[6]

```
(0.2334744405000001, 0.2706705664732254, 0.0, 0.25)
```



On remarque que pour la variable aléatoire Z , on a $Z(\Omega) = \mathbb{N}^*$.

4.6.2 Lois continues

4.6.2.1 Création de v.a.r.

In[7]

```
1 from scipy.stats import uniform, expon, norm, chi2
```

In[8]

```

1 # Création d'une loi uniforme !
  ↳ \textdbend!
2 A = uniform(1, 2)

```

In[9]

```

1 # Création d'une loi exponentielle !
  ↳ \textdbend!
2 B = expon(scale=1/2)

```

In[10]

```

1 # Création d'une loi normale ! $\mathcal{N}(\text{left}(2,$ 
  ↳  $3^2\text{right})$  \textdbend!
2 C = norm(2, 3)

```

In[11]

```

1 # Création d'une loi du ! $\chi^2$ ! à 4 degrés de liberté
2 D = chi2(4)

```

4.6.2.2 Utilisation particulière des lois absolument continues

In[12]

```

1 # Densité
2 B.pdf(1), C.pdf(2)

```

Out[12]

```
(0.2706705664732254, 0.1329807601338109)
```

4.6.3 Fonctions communes

In[13]

```
1 # Fonctions de répartition
2 A.cdf(2), D.cdf(6), T.cdf(5.5)
```

Out[13]

```
(0.5, 0.8008517265285442, 0.5)
```

In[14]

```
1 # Quantiles
2 norm().ppf(0.95), T.ppf(0.75)
```

Out[14]

```
(1.6448536269514722, 6.0)
```

Remarquer le quantile fourni pour les lois discrètes...

In[15]

```
1 # Simulation
2 B.rvs(size=50)
```

Out[15]

```
array([0.50172448, 0.81312834, 1.32932505, 1.30934829, 0.21552946,
        1.10234946, 0.30723986, 1.59033735, 0.18432886, 0.32891211,
        0.61231896, 0.19666015, 0.09321    , 0.55683726, 0.48993168,
        0.36560171, 1.36419851, 0.2636374 , 0.03107481, 0.50694319,
        0.31077924, 0.917985  , 0.56237205, 0.0971847 , 0.82811461,
        0.9536306 , 0.75348393, 0.78003073, 0.01020769, 0.3114689 ,
        0.37869123, 0.39866172, 0.44022813, 0.06442999, 0.78394273,
        0.31729352, 0.24430013, 0.00589353, 0.33997101, 0.84190293,
        0.57899009, 0.09054066, 0.13232596, 0.31005055, 0.27825983,
        0.61011195, 1.14595666, 0.52139951, 0.2490231 , 0.93866074])
```

In[16]

```
1 # Moyenne, écart-type, médiane  
2 C.mean(), C.std(), C.median()
```

Out[16]

```
(2.0, 3.0, 2.0)
```